

有限オートマトン（DFA）の状態遷移図を機械的に作る手順

「白紙から状態遷移図が作れない」を解消するための型 — 千葉大・専門科目 対策ノート

大原則（これ1つが全部の土台）：
状態とは「最後に合格／不合格を判定するために、途中で覚えておくべき情報」のこと。
作図でやることは結局たった1つの問いに答えるだけ — 「判定に必要な"覚えておくべきこと"は何で、それは何通りあるか？」
それが有限通りなら、その通り数が状態数。あとは矢印を埋める単純作業。

1. 作図の手順（毎回この7ステップ）

- 1. 受理条件を言い換える … 「ここまでの入力のお〇〇」という形に
- 2. 覚えるべき情報を特定 … その〇〇を判定するのに必要な最小限の情報（→ §2のカatalogで当てはめる）
- 3. 状態を列挙して名前をつける … 情報の取りうる値ぜんぶ
- 4. 開始状態を決める … 空文字列（何も読んでいない）ときの値
- 5. 受理状態を選ぶ … 条件を満たす値を、状態の言葉に翻訳
- 6. 遷移を埋める … 各状態 × 各文字 で行き先を機械的に書く（ここは作業）
- 7. 具体例で検算 … 短い文字列を数本流して合否を確認

難しいのは 2 だけ。そこさえ決まれば 3～7 はほぼ作業です。

2. 「覚えること」パターンカatalog（＝武器）

作図がひらめきに見えるのは、このカatalogを持っていないから。覚えるべき情報はほぼ5パターン。条件を見てどれかに当てはめます。

条件のタイプ	覚える情報（＝状態の正体）	状態数
「〇の個数が偶数／奇数」	パリティ（偶 or 奇）	2
「〇の個数が m の倍数」「mod m 」	m で割った余り	m
「△△で終わる」	直近の数文字（スライド窓）	少数
「△△という並びを含む」	パターンを今どこまで一致できたか（進捗）	パターン長+1
「△△で始まる」	最初の判定が済んだか（進捗）	少数
（無限に増える量を覚える必要があるもの）	DFAでは不可能のサイン（§7）	∞

複合条件（「AかつB」「AまたはB」）は、A用とB用の機械を別々に作って組み合わせる（§6 積構成）。

3. 例で身につける

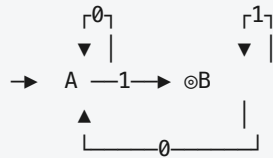
例1：「1 で終わる」なら合格

覚えること：「最後に読んだ文字が 1 か否か」だけ。→ 2状態。

状態A：最後の文字が 0（または何も読んでいない） ... 不合格
状態B：最後の文字が 1 ... 合格

A $\xrightarrow{0}$ A A $\xrightarrow{1}$ B
B $\xrightarrow{0}$ A B $\xrightarrow{1}$ B

開始：A 受理：B



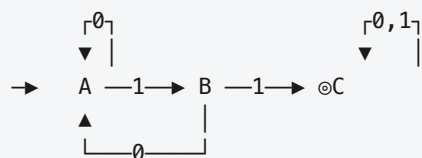
例2：「11 を含む」なら合格

覚えること：「今、1 が何個連続しているか（0個／1個／もう11が出た）」。→ 3状態。

A：1の連続0個 B：1の連続1個 C：もう11が出た（合格確定）

A $\xrightarrow{0}$ A A $\xrightarrow{1}$ B
B $\xrightarrow{0}$ A B $\xrightarrow{1}$ C
C $\xrightarrow{0}$ C C $\xrightarrow{1}$ C ← 一度Cに入ったら出ない（吸収状態）

開始：A 受理：C



「もう条件を満たした」「もう二度と満たせない」状態は、自分へのループ（吸収状態）になりがち。

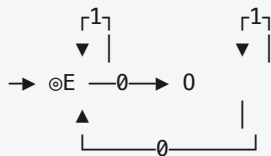
例3：「0 が偶数個」なら合格

覚えること：「これまでの 0 の個数のパリティ（偶 or 奇）」。1 の個数は関係ないので無視。→ 2 状態。

E : 0 が偶数個 0 : 0 が奇数個

E $\xrightarrow{0}$ E E $\xrightarrow{1}$ 0 ← 0 で反転、1 では変わらない
0 $\xrightarrow{0}$ 0 0 $\xrightarrow{1}$ E

開始：E (0 個=偶数) 受理：E



4. 例4：「 m の倍数」を判定する（2進数）

「2進数として読んで m の倍数なら合格」。これは **mod m を追う**（カタログ2行目）。状態 = 「 m で割った余り」で m 個。

核心トリック：2進数を左の桁から読むと、1文字読むごとに値はこう変わる：

新しい値 = (今の値) $\times$ 2 + (読んだ文字 0か1)

理由：桁が1つ左にずれる=2倍、末尾に新しい文字が入る。

欲しいのは余りだけなので、毎回「**余りを2倍して、読んだ文字を足し、また m で割った余り**」を見ればよい。

例4a：2 の倍数（簡単）

2倍は必ず偶数なので、余りは**最後のビット**だけで決まる → 末尾が 0 なら合格。実質1ビット判定。

例4b：3 の倍数（余りを追う基本形）

状態 = 余り $\{0, 1, 2\}$ 。各余りで「2倍して+文字、3で割った余り」を計算して埋める：

余り0: 0を読む $\rightarrow(0 \cdot 2+0)=0 \rightarrow$ 余り0 1を読む $\rightarrow(0 \cdot 2+1)=1 \rightarrow$ 余り1
余り1: 0を読む $\rightarrow(1 \cdot 2+0)=2 \rightarrow$ 余り2 1を読む $\rightarrow(1 \cdot 2+1)=3 \rightarrow$ 余り0
余り2: 0を読む $\rightarrow(2 \cdot 2+0)=4 \rightarrow$ 余り1 1を読む $\rightarrow(2 \cdot 2+1)=5 \rightarrow$ 余り2

r0 $\xrightarrow{0}$ r0 r0 $\xrightarrow{1}$ r1 開始：r0 受理：r0 (余り0=3で割り切れる)
r1 $\xrightarrow{0}$ r2 r1 $\xrightarrow{1}$ r0
r2 $\xrightarrow{0}$ r1 r2 $\xrightarrow{1}$ r2

検算: "11" (=3) $\rightarrow$ r0 $\rightarrow$ r1 $\rightarrow$ r0 合格◎ / "101" (=5) $\rightarrow$ r0 $\rightarrow$ r1 $\rightarrow$ r2 $\rightarrow$ r2 不合格

注意：「1を読む=余り+1」ではない。足す前に必ず全体が2倍されるため。2の倍数が末尾だけで済むのは特別ラッキーなだけで、3以上はこの「余りを追う」が普通のやり方。

5. 複合条件「A かつ／または B」 = 積構成

条件が2つあるときは、**A用の機械とB用の機械を別々に作って、同時に走らせる**。状態は組 (Aの状態, Bの状態)。

条件	受理状態の決め方
A かつ B	Aもbも受理のとき受理
A または B	どちらか一方でも受理なら受理
A でない (補集合)	受理／非受理をそっくり入れ替える

例：「2の倍数 または 3の倍数」 … $A=\text{mod}2$ (2状態)、 $B=\text{mod}3$ (3状態) → 組で $2 \times 3 = 6$ 状態。2と3をまとめると $\text{mod} 6$ と同じ (最小公倍数)。状態 $q_0 \sim q_5 = \text{値 mod } 6$ 、遷移は「2倍して+文字、6で割った余り」。受理 = 「2で割れる $\{0, 2, 4\}$ 」 $\cup$ 「3で割れる $\{0, 3\}$ 」 = $\{0, 2, 3, 4\}$ 、拒否は $\{1, 5\}$ だけ。

```

q0 -0→q0 -1→q1  ⊙      q3 -0→q0 -1→q1  ⊙
q1 -0→q2 -1→q3          q4 -0→q2 -1→q3  ⊙
q2 -0→q4 -1→q5  ⊙      q5 -0→q4 -1→q5
開始 q0   受理 {q0,q2,q3,q4}

```

6. 「DFAでは無理」を見抜く

いくら状態を増やしても足りない条件もある。見分け方：

「無限に増え続ける量を、最後まで正確に覚える必要がある」なら DFA では不可能。

```

「0の個数 = 1の個数」  → 差を無限に覚える必要 → 不可能
「括弧が正しく対応」  → ネストの深さが無限   → 不可能 (PDAの領域)
「0^n1^n」             → n を覚える必要       → 不可能

```

パリティや mod のように**有限の値に圧縮できれば可能**、無限の量を覚えるなら不可能。(後者はスタックを持つプッシュダウンオートマトン以上が必要。ただし千葉大の過去問は有限オートマトンの範囲のみ。)

7. 詰まったら自分に聞く3問 (チェックリスト)

1. 受理条件を「ここまでの入力 of 何の性質」で言える？
2. その性質を判定するのに覚える情報は？ 有限通り？
→ カタログのどれ？ (パリティ / mod / 末尾 / 進捗) / 複合なら積構成に分解できる？
3. 有限に圧縮できないなら → そもそもDFA不可能では？ (§6)

作る力 = 「何を覚えるか」を決める (パターン当てはめ) + 矢印を引く (作業)。前者だけが本当の山場で、それもカタログで大半は機械化できる。